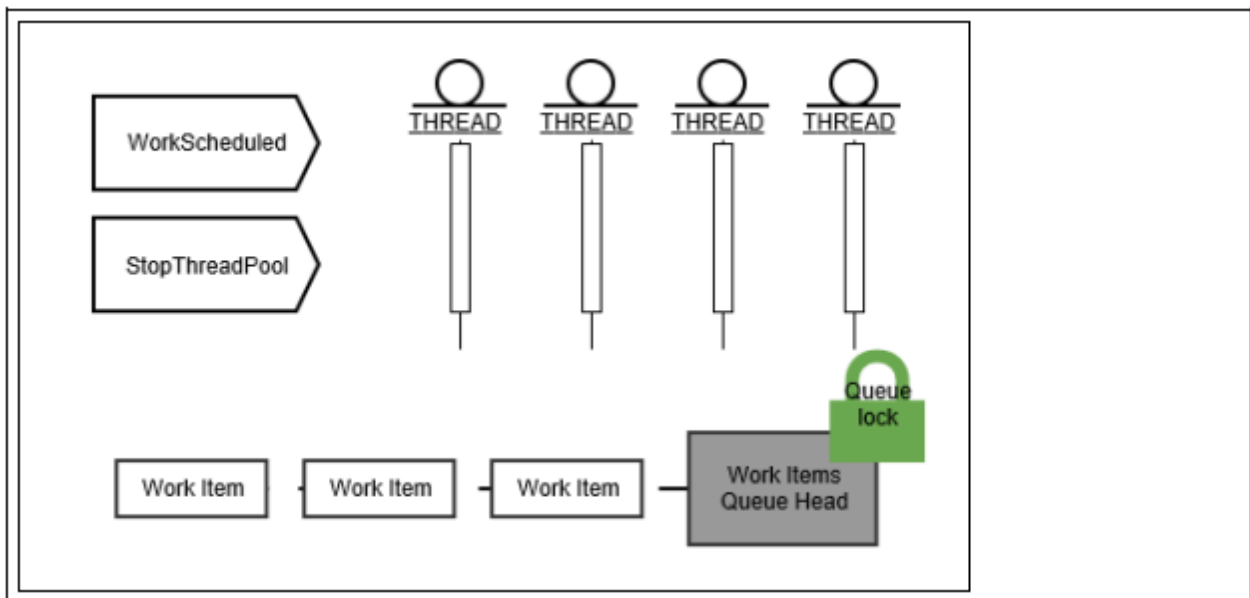


Thread Pool

The role of a thread pool is to simply execute Work Items (simple functions) on pre-created threads. The architecture of a thread pool reduces to a thread safe queue and a set of threads. The threads are controlled by two control signals

- WorkScheduledEvent → signals that work needs to be performed;
- StopThreadPoolEvent → signals that the threads in the thread pool stop;



In a Windows application, the two aforementioned events are represented as a HANDLE to an EVENT structure. The list of work items can be represented by a doubly linked list protected by a lock (SRWLOCK / MUTEX / ...).

- To learn how you can use events please see the following link: <https://learn.microsoft.com/en-us/windows/win32/sync/using-event-objects> .
- To learn more about Slim Reader/Writer (SRW) locks see the following link: <https://learn.microsoft.com/en-us/windows/win32/sync/slim-reader-writer--srw--locks> .
- To learn more about Doubly Linked Lists see <https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/singly-and-doubly-linked-lists>

You're also given a threadpool.c file which contains a starting point which you can use in your assignment. Below is a description on how to use the APIs. The threadpool is not complete it has *todos* throughout the project. If you decide to use them it is your responsibility to implement them. You are welcomed to add any additions to the APIs to make them more secure from a code security point of view.

To initialize the threadpool, you may call

```
NTSTATUS
TpInit(
```

```

    _Inout_ MY_THREAD_POOL* ThreadPool,
    _In_ UINT32 NumberOfThreads
)

```

which will create a threadpool with the given number of threads.

To destroy a previously created threadpool, you may use

```

void
TpUninit(
    _Inout_ MY_THREAD_POOL* ThreadPool
)

```

which will wait for all threads to finish.

To enqueue a work item to be executed asynchronously, you may use

```

NTSTATUS
TpEnqueueWorkItem(
    _Inout_ MY_THREAD_POOL* ThreadPool,
    _In_ LPTHREAD_START_ROUTINE WorkRoutine,
    _In_opt_ PVOID Context
);

How to call:
    status = TpEnqueueWorkItem(&tp, TestThreadPoolRoutine, &ctx);

```

The **WorkRoutine** parameter can be declared like:

```

DWORD WINAPI
TestThreadPoolRoutine(
    _In_opt_ PVOID Context
)

```

The **Context** is a caller defined context, and is optional, meaning you can pass NULL if you do not want to use it.

Note: This thread pool does not distribute work equally. If multiple threads enqueue work before a thread dequeues and executes work items, due to the nature of the synchronization event other threads will not wake up to handle the work items. Lots of improvements can be made to this implementation. It is provided only for you to understand how it internally work.